

Raytracer Project Report

Computer Graphics Labs 1–4

Youssef Naji

Abstract

This report describes the implementation of a small physically based raytracer developed over four lab sessions. The final renderer supports diffuse and mirror surfaces, direct lighting with hard shadows from a point light source, recursive indirect illumination, stochastic anti-aliasing, triangle mesh rendering with Moller–Trumbore intersection, and acceleration using a bounding volume hierarchy. The final scene renders an OBJ cat mesh inside a Cornell-box-like environment made of large colored spheres.

May 20, 2026

1 Overview of the final renderer

The goal of the project was to progressively build a raytracer from basic geometric visibility to a small path tracer. The final code represents 3D vectors, rays, abstract objects, spheres, triangle meshes, and a scene containing all objects. Each pixel is rendered by sending one or several camera rays through the image plane. The closest object intersection determines the visible surface point, its normal, and the material response. The scene used for the final tests contains a cat OBJ mesh, six large spheres used as colored walls, a point light source, and a camera located in front of the scene.

The main rendering equation was approximated recursively. For a diffuse surface, the color is the sum of a direct lighting term and an indirect term sampled over the hemisphere around the normal. For mirror surfaces, the outgoing ray is replaced by the perfect reflection direction

$$\omega_r = \omega_i - 2(\omega_i \cdot n)n.$$

The implementation also includes a boolean flag for transparent materials in the object class, but the submitted version focuses on diffuse and mirror behavior.

Expected final rendered image

Insert the final `image.png` produced by the code here. It should show the cat mesh inside the colored room, with shadows, indirect light, and anti-aliasing. In the submitted code the image size is 1000x1000, the number of samples per pixel is 32, and the maximum recursion depth is 20.

Figure 1: Expected final rendered image

2 Lab 1: sphere raytracing, direct lighting, shadows, and mirrors

The first lab established the base of the raytracer. I implemented ray-sphere intersection by substituting the parametric ray equation

$$P(t) = O + tu$$

into the sphere equation $\|P - C\|^2 = R^2$. This gives a quadratic equation in t . The code keeps the smallest positive solution, rejects negative intersections behind the camera, computes the hit point P , and normalizes the geometric normal $N = (P - C)/R$. This was one of the first debugging points: the image can become black or incorrect if negative values of t are accepted, if the normal is not normalized, or if the camera ray direction is wrong.

I then implemented scene intersection by looping over all objects and keeping the closest positive hit. This made it possible to render several spheres at once and to correctly handle occlusion. Direct lighting was added for a point light source using an inverse-square attenuation factor. The diffuse material follows Lambert's law: the visible contribution is proportional to $\max(0, N \cdot \omega_l)$. I also implemented shadow rays by casting a ray from the hit point toward the light. If another object is found before the light distance, the point is in shadow and the direct contribution is removed.

Mirror surfaces were implemented recursively. When a ray hits a mirror object, the renderer computes the reflected direction and calls `getColor` again. A small offset $10^{-6}N$ is added to the ray origin to avoid self-intersection artifacts, also called shadow acne. The recursion is bounded using `max_light.bounce`, which prevents infinite reflection chains.

3 Lab 2: indirect lighting and anti-aliasing

The second lab added the Monte Carlo part of the renderer. Instead of only computing direct light from the point light source, each diffuse intersection launches a random secondary ray over the hemisphere

centered around the surface normal. The random direction is generated proportionally to the cosine factor. In the code, two random variables $r_1, r_2 \in [0, 1]$ are used:

$$\cos \theta = \sqrt{1 - r_2}, \quad \sin \theta = \sqrt{r_2}, \quad \phi = 2\pi r_1.$$

An orthonormal basis (u, v, N) is constructed around the normal, then the local sample direction is transformed into world coordinates. This was an important implementation detail because sampling directly in global coordinates would not align the hemisphere with the surface normal.

The indirect contribution is estimated by recursively evaluating the color in the sampled direction and multiplying it by the object's albedo component by component. Russian roulette was not used, as required by the assignment. Instead, recursion terminates when the maximum bounce count is reached. In the final code, the maximum is set to 20, which is higher than the example in the instructions and improves global illumination at the cost of render time.

Anti-aliasing was added by jittering the primary ray within each pixel. Instead of shooting a ray exactly through the pixel center, the code adds random offsets u and v to the pixel coordinates and averages 32 samples per pixel. This reduces jagged edges on silhouettes and high-contrast boundaries, especially around the cat mesh. A practical challenge here was noise: few samples per pixel render fast but produce grainy indirect lighting, while more samples improve convergence but increase computation time almost linearly.

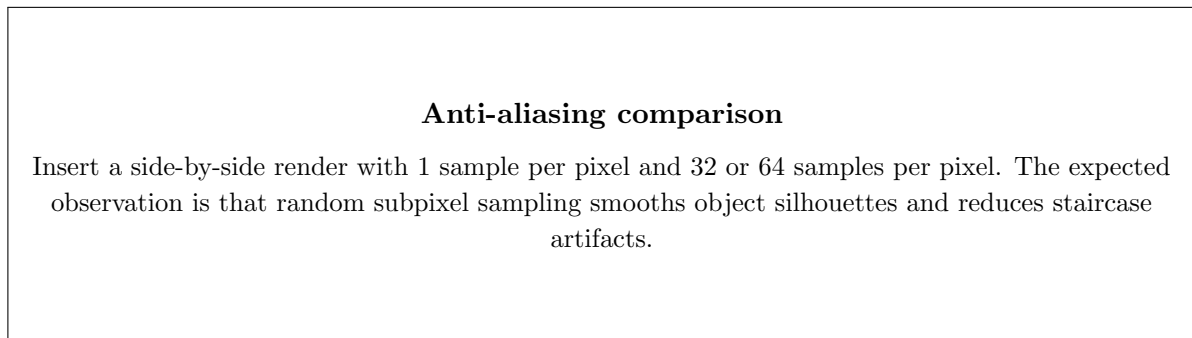


Figure 2: Anti-aliasing comparison

4 Lab 3: triangle mesh rendering with Moller–Trumbore

The third lab replaced simple analytic spheres with a triangle mesh. The code reads an OBJ file, stores vertices, texture coordinates, normals, and triangle indices, then renders the cat mesh. The intersection routine uses the Moller–Trumbore algorithm. For a triangle with vertices A, B, C , a ray intersects the triangle when

$$O + tu = A + \alpha(B - A) + \beta(C - A),$$

with $t > 0$, $\alpha \geq 0$, $\beta \geq 0$, and $\alpha + \beta \leq 1$. The implementation computes the barycentric coordinates using cross products and rejects intersections outside the triangle. If vertex normals are available, the normal is interpolated barycentrically; otherwise the geometric face normal is used.

A useful debugging point was the orientation of the normal. If the interpolated normal faces in the same direction as the ray, the code flips it. This keeps lighting stable and avoids surfaces being lit from the wrong side. Another challenge was performance. Testing every triangle for every ray is too slow for a full image, especially after adding indirect bounces and multiple samples per pixel. Lab 3 therefore prepared the transition to acceleration structures by isolating triangle intersection and bounding-box tests.

Mesh rendering result

Insert a render of the cat mesh without BVH or with only simple bounding-box acceleration. This image illustrates the transition from analytic spheres to arbitrary triangle geometry.

Figure 3: Mesh rendering result

5 Lab 4: BVH acceleration

The fourth lab added a bounding volume hierarchy for the triangle mesh. Each BVH node stores an axis-aligned bounding box, a range of triangles, and either two child nodes or a leaf marker. The construction first computes the bounding box of the current triangle range. If the range contains at most eight triangles, a leaf is created. Otherwise, triangle centroids are computed, the largest centroid extent determines the split axis, and `std::nth_element` partitions the triangles around the median. This produces a balanced hierarchy without fully sorting all triangles at each level.

During intersection, the ray is first tested against the node bounding box using slab intersection. If the ray misses the box, all triangles inside the node are skipped. If it hits a leaf, only the few triangles in that leaf are tested with Moller–Trumbore. If it hits an internal node, both children are visited recursively while the current closest distance t is passed down. This is the key optimization of the project: instead of testing every triangle for every ray, the renderer eliminates large groups of triangles quickly.

In practice, BVH support was necessary because the renderer combines triangle meshes, recursive lighting, and many samples per pixel. Without BVH, the render time becomes dominated by ray-triangle intersection. With BVH, the final code can render the cat scene at 1000x1000 resolution with 32 samples per pixel. The assignment statement reports that similar scenes can go from several minutes without BVH to seconds or tens of seconds with BVH depending on hardware. My implementation follows the same idea by building the hierarchy once after loading and transforming the mesh, then reusing it for all ray queries.

BVH performance illustration

Insert a small table or plot comparing render time without BVH, with a single bounding box, and with the BVH. Also indicate the number of triangles in the cat mesh printed by the program.

Figure 4: BVH performance illustration

6 Implementation choices and challenges

Several details were important for correctness. First, all ray directions and normals must be normalized before being used in dot products. Second, secondary rays must be offset from the surface by a small epsilon to avoid immediately hitting the same surface. Third, the OBJ parser had to handle several face formats, including vertex-only, vertex/UV, vertex//normal, and vertex/UV/normal indices. Fourth, OpenMP was used to parallelize the pixel loop, but this required one random generator per thread to avoid sharing a single generator across threads.

The main practical challenge was the tradeoff between quality and time. Increasing samples per pixel improves anti-aliasing and reduces Monte Carlo noise, while increasing the maximum bounce count improves indirect lighting. However, both increase the number of recursive rays. The BVH was therefore not just an optional optimization, but the feature that made the final path-traced mesh scene usable.

7 Conclusion

Across the four labs, the renderer evolved from a simple sphere raycaster into a recursive Monte Carlo raytracer capable of rendering triangle meshes efficiently. Lab 1 built the geometric and lighting foundations, Lab 2 added stochastic integration and anti-aliasing, Lab 3 introduced OBJ mesh rendering through ray-triangle intersection, and Lab 4 made the mesh renderer practical using a BVH. The final code demonstrates the main components of a physically based renderer: visibility, material response, shadow testing, recursive light transport, sampling, and acceleration structures.